



How to accelerate Python with GPUs – made simple

03.11.2025

-

Geray Karademir

GPU offloading overview

GPU offloading in Python

- Low level APIs:
 - E.g. PyCUDA, PyOpenCL

Pro	Cons
- Maximum performance and control - Full access to GPU memory, kernel code, and device features - Fine-grained tuning for performance-critical kernels	- Verbose and error-prone (hard to debug) - Requires knowledge of GPU architecture, thread hierarchy, and memory management - Steep learning curve

GPU offloading in Python

- Mid level APIs:
 - E.g. PySYCL, Triton (OpenAI)

Pro	Cons
- Can still achieve near-native CUDA performance - Automatic JIT compilation - More readable, fewer lines of code	- Some hardware features not exposed - Limited portability (e.g. Triton = NVIDIA GPUs only).

GPU offloading in Python

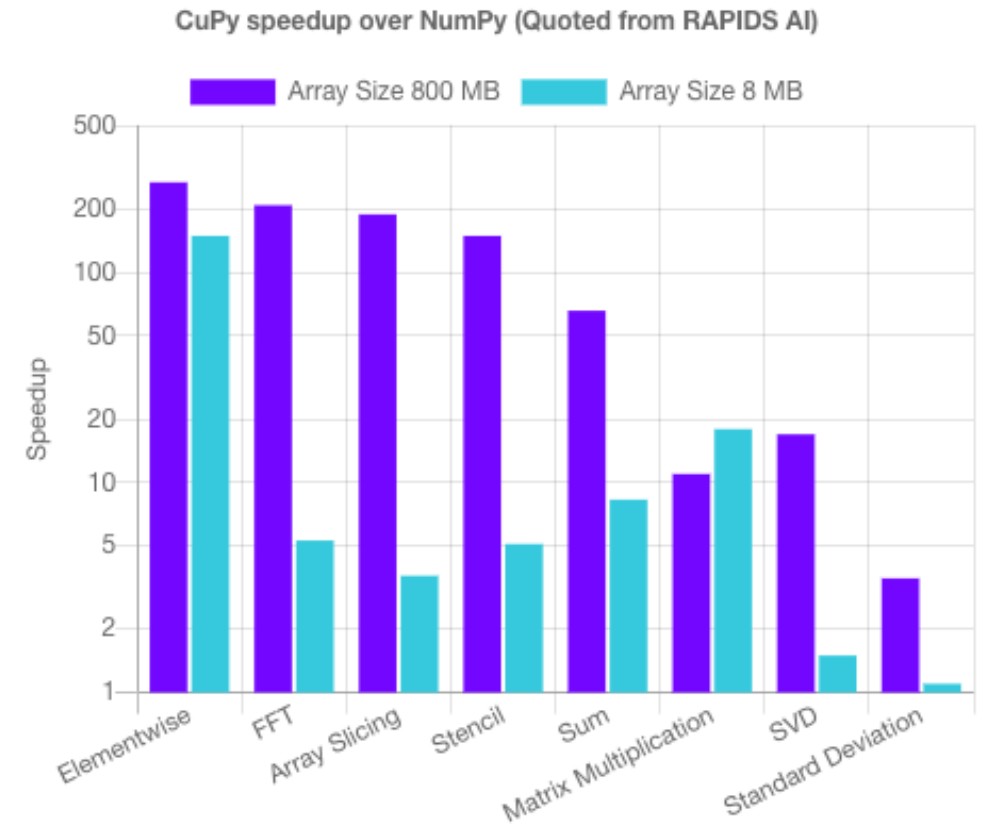
- High level APIs:
 - E.g. JAX

Pro	Cons
- Automatic GPU acceleration (no kernel writing)	- Limited low-level control. - Hard to optimize beyond framework defaults

But what about very easy?

CuPy (<https://cupy.dev/>)

- CuPy is an open-source array library for GPU-accelerated computing with Python.
- Highly compatible with NumPy and SciPy; in most cases it can be used as a drop-in replacement.
- Availability of custom CUDA kernels.



CuPy – available tools

- **N-dimensional array** (`ndarray`): `cupy.ndarray`
 - Data types (dtypes): boolean (`bool_`), integer (`int8`, `int16`, `int32`, `int64`, `uint8`, `uint16`, `uint32`, `uint64`), float (`float16`, `float32`, `float64`), and complex (`complex64`, `complex128`)
 - Supports the semantics identical to `numpy.ndarray`, including basic / advanced indexing and broadcasting
- **Sparse matrices**: `cupyx.scipy.sparse`
 - 2-D sparse matrix: `csr_matrix`, `coo_matrix`, `csc_matrix`, and `dia_matrix`
- **NumPy Routines**
 - Module-level Functions (`cupy.*`)
 - Linear Algebra Functions (`cupy.linalg.*`)
 - Fast Fourier Transform (`cupy.fft.*`)
 - Random Number Generator (`cupy.random.*`)
- **SciPy Routines**
 - Discrete Fourier Transforms (`cupyx.scipy.fft.*` and `cupyx.scipy.fftpack.*`)
 - Advanced Linear Algebra (`cupyx.scipy.linalg.*`)
 - Multidimensional Image Processing (`cupyx.scipy.ndimage.*`)
 - Sparse Matrices (`cupyx.scipy.sparse.*`)
 - Sparse Linear Algebra (`cupyx.scipy.sparse.linalg.*`)
 - Special Functions (`cupyx.scipy.special.*`)
 - Signal Processing (`cupyx.scipy.signal.*`)
 - Statistical Functions (`cupyx.scipy.stats.*`)

CuPy – How to install and use at Physik @ LMU

- **Note: to install cupy it is important to know the cuda version available on your system!!!**

E.g. on Physik @ LMU:

- `ml cuda/12.4.0`

And then in your venv:

- `pip install cupy-cuda12x`

- Done!

```
#!/bin/bash -l
#SBATCH --job-name=Powerspec
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=3
#SBATCH --cpus-per-task=2
#SBATCH --gres=gpu:h100nvl:2
#SBATCH --partition=inter
#SBATCH --mem=100GB
#SBATCH --time=05:00:00
#SBATCH --output=Powerspec.%j.out

ml purge
ml spack/2024.07
ml cuda/12.4.0
ml openmpi/5.0.0-gcc-11.4.1-cuda12.4

source venv/LMUPhysikPowerspecVenv/bin/activate

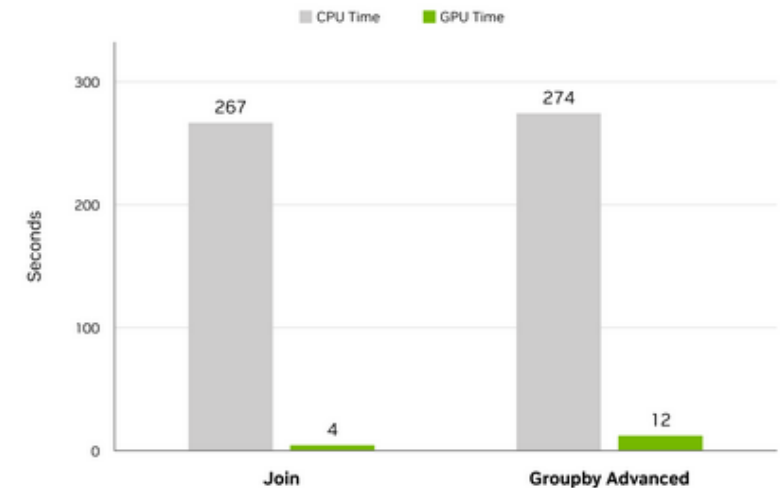
mpiexec -np $SLURM_NTASKS python Powerspec.py
```

cuDF (<https://developer.nvidia.com/topics/ai/data-science/cuda-x-data-science-libraries/cudf>)

- GPU acceleration for pandas.
- Integration in libraries like CuPy, Numba and scikit-learn.
- Checkout free cuDF workshop by NVIDIA:
https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+T-DS-01+V1
- For all accelerated libraries by NVIDIA see:
<https://developer.nvidia.com/topics/ai/data-science/cuda-x-data-science-libraries>

From Minutes to Seconds: cuDF Accelerates pandas

Standard DuckDB data benchmark (5 GB) performance comparison between cudf.pandas and traditional pandas v2.2 (lower is better).



Specs: HW: NVIDIA L4, CPU: Intel Xeon 8480CL | SW: pandas v2.2.1, NVIDIA RAPIDS™ cuDF 24.02

Dpnp (<https://intelpython.github.io/dpnp/index.html>)

- Library by Intel (SYCL) that implements a subset of NumPy that can be executed on any data parallel device (similar to CuPy)
- Limited support for SciPy

Summary

Name	NumPy	DPNP	CuPy
Module-Level	391	322	330
Multi-Dimensional Array	53	42	47
Linear Algebra	31	31	16
Discrete Fourier Transform	18	18	18
Random Sampling	53	48	49
Total	546	461	460

Lets have a look at the example!